

Braverge Technical Report
Formal Specification of the Braverge Change
Operation Syntax (BCOS) and Branch
Metadata Schema (BMS)

Mohammad-Sajad Kasaei
`smskasaei@gmail.com`

June 18, 2026

Contents

1	Introduction	3
2	Design Objectives	3
3	Repository Architecture	4
4	Branch Metadata Schema (BMS)	4
4.1	Purpose	4
4.2	Formal Specification	5
4.3	Identifier	5
4.4	Origin	5
4.5	Collaborators	5
4.6	LifecyclePolicy	6
4.7	Scope	6
4.8	VersionSet	6
5	Branch Relationship Management	7
6	Braverge Change Operation Syntax (BCOS)	7
6.1	Purpose	7
6.2	Operation Categories	7
6.3	Formal Specification	8
7	ADD Operations	8
7.1	ElementCreation	8
8	DELETE Operations	9
8.1	ElementRemoval	9
9	UPDATE Operations	9
9.1	PropertyModification	9
10	Version Reconstruction	9
11	BCOS and Branching	10
12	BCOS and Merging	10
13	Traceability Support	10
14	Extensibility	10
15	Conclusion	11

1 Introduction

Collaborative model evolution requires mechanisms capable of tracking model modifications, preserving historical information, supporting branching strategies, and enabling consistent model integration. While traditional version-control systems manage textual artifacts through line-based differencing and merging, model-based artifacts require representations that capture structural and semantic modifications at the model level.

To support these requirements, Braverge introduces two complementary specifications:

- **Braverge Change Operation Syntax (BCOS)**
- **Branch Metadata Schema (BMS)**

BCOS provides a technology-independent representation of model modifications, while BMS stores branch-level information required for collaboration, synchronization, traceability, and lifecycle management.

Together, BCOS and BMS constitute the persistence layer of Braverge and provide the foundation for version reconstruction, branching, merging, synchronization, and collaborative model evolution.

2 Design Objectives

The design of BCOS and BMS is guided by the following objectives:

1. Tool independence.
2. Technology neutrality.
3. Storage efficiency.
4. Fine-grained traceability.
5. Branch-aware collaboration.
6. Extensibility.
7. Support for distributed development.

Rather than storing complete copies of every model version, Braverge stores the most recent model version together with a history of change operations. This hybrid strategy combines the advantages of state-based and operation-based version management.

3 Repository Architecture

Each branch repository maintained by Braverge contains three primary artifacts:

1. Branch Metadata Schema (BMS)
2. History Evolution (BCOS Operations)
3. Newest Model Version

Figure 1 illustrates the relationship among these artifacts.

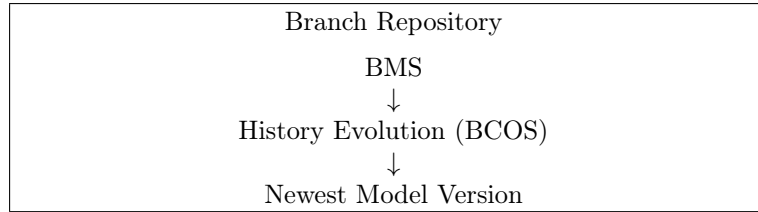


Figure 1: Repository architecture.

The newest version represents the current state of the model. BCOS records the sequence of modifications that produced this state. BMS stores metadata required to manage the branch itself.

4 Branch Metadata Schema (BMS)

4.1 Purpose

BMS captures information describing branch characteristics independently of the model content.

The schema supports:

- branch identification
- ancestry management
- collaborator management
- lifecycle management
- access control
- version traceability

4.2 Formal Specification

```
{
  "Person": {
    "UserId": "String"
  },

  "Version": "String",

  "BMS": {
    "Identifier": "String",
    "Origin": "String",
    "Collaborators": ["Person"],
    "LifecyclePolicy": "String",
    "Scope": "Person",
    "VersionSet": ["Version"]
  }
}
```

4.3 Identifier

The *Identifier* field uniquely identifies a branch within the repository.

Example:

```
"Identifier": "SubsystemA"
```

The identifier must be unique throughout the repository.

4.4 Origin

The *Origin* field records the parent branch from which the current branch was created.

Example:

```
"Origin": "MASTER"
```

This field enables reconstruction of branch ancestry and supports synchronization and merge analysis.

4.5 Collaborators

The *Collaborators* field stores all users authorized to access the branch.

Example:

```
"Collaborators": [
  {"UserId": "PE"},
  {"UserId": "SM"}
]
```

This information is used by the collaboration layer to enforce access-control policies.

4.6 LifecyclePolicy

Lifecycle policies determine branch persistence.

Supported values include:

- Permanent
- Temporary

Permanent branches persist indefinitely, whereas temporary branches are intended for short-lived development activities and may be removed after integration.

4.7 Scope

Branches can be scoped to the entire model or restricted to specific views.

Possible values include:

- full: full access to the entire model
- partial: restricted access limited to specific views or collaborators

Scope information is used by the access-control mechanism.

4.8 VersionSet

VersionSet contains all versions belonging to the branch.

Example:

```
"VersionSet": [
  "V1",
  "V2",
  "V3",
  "V4"
]
```

VersionSet supports:

- rollback
- auditing
- traceability
- reconstruction

5 Branch Relationship Management

Branch relationships are represented through the Origin attribute.

Example hierarchy:

```
MASTER
|-- SubsystemA
|   |-- FeatureA
|   '-- FeatureB
'-- SubsystemB
```

Maintaining explicit branch ancestry supports:

- merge preparation
- synchronization
- conflict analysis
- historical reconstruction

6 Braverge Change Operation Syntax (BCOS)

6.1 Purpose

BCOS provides a technology-independent representation of model evolution.

Instead of storing complete model states, Braverge records a sequence of model modifications.

The syntax supports:

- version reconstruction
- differencing
- merging
- synchronization
- auditing

6.2 Operation Categories

BCOS contains three primitive operation categories:

1. ADD
2. DELETE
3. UPDATE

These operations are sufficient to represent arbitrary model evolution.

6.3 Formal Specification

```
{
  "BCOS": {
    "ADD": ["ElementCreation"],
    "DELETE": ["ElementRemoval"],
    "UPDATE": ["PropertyModification"]
  },

  "ElementCreation": {
    "elementId": "String"
  },

  "ElementRemoval": {
    "elementId": "String",
    "parentId": "String",
    "elementMetadata": "String",
    "elementReferences": ["Reference"]
  },

  "Reference": {
    "name": "String",
    "targetId": "String"
  },

  "PropertyModification": {
    "elementId": "String",
    "operationType": "String",
    "modificationType": "String",
    "name": "String",
    "value": "String"
  }
}
```

7 ADD Operations

7.1 ElementCreation

The ElementCreation operation records the creation of a model element.

```
{
  "elementId": "Sensor01"
}
```


8 DELETE Operations

8.1 ElementRemoval

Deletion requires additional information because the removed element may be needed during merge and rollback operations.

```
{
  "elementId": "Sensor01",
  "parentId": "Controller01",
  "elementMetadata": "EClass",
  "elementReferences": []
}
```

Stored information enables:

- rollback
- conflict detection
- merge analysis
- historical reconstruction

9 UPDATE Operations

9.1 PropertyModification

PropertyModification captures changes applied to existing elements.

```
{
  "elementId": "Sensor01",
  "operationType": "SET",
  "modificationType": "ATTRIBUTE",
  "name": "threshold",
  "value": "75"
}
```

10 Version Reconstruction

Any version can be reconstructed using:

$$Version_i = \text{Replay}(BCOS_i, \text{NewestVersion})$$

The reconstruction mechanism eliminates the need to store complete copies of every version.

11 BCOS and Branching

When a new branch is created:

1. BMS records ancestry.
2. Version histories diverge.
3. Independent BCOS sequences are maintained.

This mechanism supports parallel model evolution.

12 BCOS and Merging

During merge operations:

1. BCOS histories are compared.
2. Common ancestors are identified.
3. Conflicting operations are detected.
4. Conflict-resolution policies are applied.
5. New BCOS operations are generated.

The resulting operation sequence becomes part of the target branch history.

13 Traceability Support

BCOS and BMS collectively provide traceability at three levels:

1. Element level
2. Version level
3. Branch level

These capabilities support auditing, impact analysis, rollback, and repository inspection.

14 Extensibility

BCOS intentionally defines a minimal set of primitive operations.

Future extensions may include:

- Move operations
- Semantic modifications

- Constraint evolution
- View synchronization operations
- AI-assisted annotations

15 Conclusion

This report presented the formal specifications of BCOS and BMS, the two foundational persistence mechanisms of Braverge. BMS captures branch-level metadata required for collaborative development, while BCOS records model evolution through technology-independent change operations. Together, these specifications provide a scalable and extensible foundation for version reconstruction, branching, merging, synchronization, conflict management, and collaborative model evolution.